```
# ----------------------------------------
# Index helper functions (0-based)
# ----------------------------------------
def left(i):
    # Step: compute the left child index.
    return 2 * i + 1


def right(i):
    # Step: compute the right child index.
    return 2 * i + 2


def parent(i):
    # Step: compute the parent index.
    if i <= 0:
        return -1
    return (i - 1) // 2


# ----------------------------------------
# Print the heap as levels
# ----------------------------------------
def print_heap_tree(arr, heap_size, title=None):
    """
    Print the heap in level order as a tree-like display.

    Shows only arr[0:heap_size] as the current heap and prints the sorted
    suffix if it exists.
    """
    # Step: optionally print a title header for clarity.
    if title:
        print(title)

    # Step: handle an empty heap display.
    if heap_size <= 0:
        print("Heap: (empty)")
        if len(arr) > 0:
            print(f"Sorted suffix: {arr}")
        print()
        return

    # Step: print level by level.
    index = 0
    level = 0
    while index < heap_size:
        # Step: count nodes on this level, capped by remaining nodes.
        level_count = min(2**level, heap_size - index)

        # Step: build a readable line for this level.
        entries = []
        for j in range(level_count):
            idx = index + j
            entries.append(f"[{idx}]={arr[idx]}")

        # Step: print the level.
        print(f"Level {level}: " + " ".join(entries))

        # Step: move to the next level.
        index += level_count
        level += 1

    # Step: print the sorted suffix, if one exists.
    if heap_size < len(arr):
        print(f"Sorted suffix: {arr[heap_size:]}")

    print()  # Blank line for spacing.


# ----------------------------------------
# Core heapify operations
# ----------------------------------------
def max_heapify(arr, i, heap_size):
    """
    Restore the max-heap property for node i within arr[0:heap_size].
```

```
    Assumes the subtrees of i are already heaps.
    """
    while True:
        # Compute child indices and find the largest among the node and
        # its children that are inside the heap.
        left_child = left(i)
        right_child = right(i)
        largest = i

        if left_child < heap_size and arr[left_child] > arr[largest]:
            largest = left_child
        if right_child < heap_size and arr[right_child] > arr[largest]:
            largest = right_child

        # If i is still the largest, the heap property holds at this node.
        if largest == i:
            break

        # Swap with the larger child and continue sifting down.
        arr[i], arr[largest] = arr[largest], arr[i]
        i = largest


def min_heapify(arr, i, heap_size):
    """Restore the min-heap property for node i within arr[0:heap_size]."""
    while True:
        left_child = left(i)
        right_child = right(i)
        smallest = i

        if left_child < heap_size and arr[left_child] < arr[smallest]:
            smallest = left_child
        if right_child < heap_size and arr[right_child] < arr[smallest]:
            smallest = right_child

        # Neither child is smaller, so the heap property holds at this node.
        if smallest == i:
            break

        # Swap with the smaller child and continue sifting down.
        arr[i], arr[smallest] = arr[smallest], arr[i]
        i = smallest


# ----------------------------------------
# Build heaps in-place
# ----------------------------------------
def build_min_heap(arr):
    n = len(arr)
    print_heap_tree(arr, n, title="Before build_min_heap (not a heap yet):")

    # Start at the last internal node and work upward to the root.
    for i in range(n // 2 - 1, -1, -1):
        min_heapify(arr, i, n)

    print_heap_tree(arr, n)
    print("Done with building MIN HEAP")


def build_max_heap(arr):
    n = len(arr)
    print_heap_tree(arr, n, title="Before build_max_heap (not a heap yet):")

    # Start at the last internal node and work upward to the root.
    for i in range(n // 2 - 1, -1, -1):
        max_heapify(arr, i, n)

    print_heap_tree(arr, n)
    print("Done with building MAX HEAP")


# ----------------------------------------
# Heapsort implementations
# ----------------------------------------
def heap_sort_max(values):
    """Return an ascending sorted copy of a list using a max heap."""
    if not isinstance(values, list):
```

```
        raise TypeError("heap_sort_max expects a list")

    arr = values.copy()
    n = len(arr)

    # Build a max heap without printing, so the sort can be used as a function.
    for i in range(n // 2 - 1, -1, -1):
        max_heapify(arr, i, n)

    # Move each maximum to the end and restore the reduced heap.
    for end in range(n - 1, 0, -1):
        arr[0], arr[end] = arr[end], arr[0]
        max_heapify(arr, 0, end)

    return arr


def heap_sort_min(values):
    """Return an ascending sorted copy of a list using a min heap."""
    if not isinstance(values, list):
        raise TypeError("heap_sort_min expects a list")

    arr = values.copy()
    n = len(arr)

    # Build a min heap without printing, so the sort can be used as a function.
    for i in range(n // 2 - 1, -1, -1):
        min_heapify(arr, i, n)

    # Moving each minimum to the end produces descending order; reverse it.
    for end in range(n - 1, 0, -1):
        arr[0], arr[end] = arr[end], arr[0]
        min_heapify(arr, 0, end)

    arr.reverse()
    return arr

import random
import time
import numpy as np
# ----------------------------
# Experiment
# ----------------------------
# 1. Create a large random array
N = 5000 # you can change size (e.g., 10_000)
arr = [random.randint(0, 10**6) for _ in range(N)]
# 2. heap Sort
arr_copy = arr[:]
start = time.perf_counter()
output = heap_sort_max(arr_copy)
t_heap = time.perf_counter() - start
print(f"Heap Sort time: {t_heap:.6f} sec")
# 3. numpy sort
arr_copy = arr[:]
start = time.perf_counter()
np_sorted = np.sort(arr_copy) # numpy's highly optimized sort (timsort/quick depending on dtype)
t_numpy = time.perf_counter() - start
print(f"NumPy Sort time: {t_numpy:.6f} sec")
```

```
Original array: [3, 9, 2, 1, 4, 5]

Before build_min_heap (not a heap yet):
Level 0: [0]=3
Level 1: [1]=9 [2]=2
Level 2: [3]=1 [4]=4 [5]=5

Level 0: [0]=1
Level 1: [1]=3 [2]=2
Level 2: [3]=9 [4]=4 [5]=5

Done with building MIN HEAP
After min heapify array: [1, 3, 2, 9, 4, 5]
Before build_max_heap (not a heap yet):
Level 0: [0]=1
Level 1: [1]=3 [2]=2
Level 2: [3]=9 [4]=4 [5]=5

Level 0: [0]=9
```

```
Level 1: [1]=4 [2]=5
Level 2: [3]=3 [4]=1 [5]=2

Done with building MAX HEAP
After max heapify array: [9, 4, 5, 3, 1, 2]

Heapsort test cases:
Lecture example: [70, 40, 60, 10, 20, 30, 50]
  Max-heap heapsort: [10, 20, 30, 40, 50, 60, 70]
  Min-heap heapsort: [10, 20, 30, 40, 50, 60, 70]
Chosen 16-number example: [42, 7, 19, 3, 88, 54, 21, 65, 12, 37, 90, 1, 28, 73, 6, 49]
  Max-heap heapsort: [1, 3, 6, 7, 12, 19, 21, 28, 37, 42, 49, 54, 65, 73, 88, 90]
  Min-heap heapsort: [1, 3, 6, 7, 12, 19, 21, 28, 37, 42, 49, 54, 65, 73, 88, 90]
Arr1: []
  Expected1: []
  Max-heap heapsort: []
  Min-heap heapsort: []
Arr2: [7, 7, 7, 7]
  Expected2: [7, 7, 7, 7]
  Max-heap heapsort: [7, 7, 7, 7]
  Min-heap heapsort: [7, 7, 7, 7]
Arr3: [5, 4, 3, 2, 1]
  Expected3: [1, 2, 3, 4, 5]
  Max-heap heapsort: [1, 2, 3, 4, 5]
  Min-heap heapsort: [1, 2, 3, 4, 5]
All heapsort examples and edge cases passed.
Heap Sort time: 0.031376 sec
NumPy Sort time: 0.002977 sec
```